

Préparation Soutenance Technique — CloudShift

Document de défense basé sur l'audit croisé CODE ↔ RAPPORT (28/06/2026)

Méthodologie : chaque affirmation ci-dessous a été vérifiée dans le code source réel (priorité absolue), puis confrontée au rapport et à la présentation. Les divergences sont signalées explicitement avec le numéro de ligne du fichier source.

0. AUDIT CODE ↔ RAPPORT — ce que le jury peut trouver

Avant tout : voici les écarts réels entre ce que tu as écrit et ce que le code fait.

Tu DOIS les connaître, parce qu'un juré qui ouvre un .py les verra.

Ce qui est EXACT et vérifiable en direct (tes points forts)

			Affirmation rap
Poids AHP 0.4253/0.2618/0.1631/0.1032/0.0467	EXACT, à 0.0001 près : 0.425322 / 0.261830 / 0.163079	code/ahpweights.py — calcul live python -c "from c	
Fallback Neo4j → SQL récursif	RÉEL : WITH RECURSIVE dep_tree(node, depth)	code/graphrag.py:348-356	
SKIP LOCKED multi-worker	RÉEL	executor/queue.py:102	
Checkpointing + reprise + double-checked lock	RÉEL : graphlock = threading.Lock() + interrupt	pipeline/pipeline_graph.py:228-240	
Classificateur d'erreurs déterministe 7 classes	RÉEL : STATIC/SEMANTIC/DEPENDENCY/RUNTIME/UNKNOWN	code/errortypes.py	

Incohérences à ASSUMER (prépare la réponse — ne pas être pris au dépourvu)

INCOHÉRENCE #1 — Les seuils 7R : trois versions différentes dans le code lui-même.

- Constantes : REHOSTHIGH=0.97, REHOSTLOW=0.94, REPLATFORMHIGH=0.94, REPLATFORMCERTAINLOW=0.86, REPLATFORMLOW=0.82 (scoringdecision_tools.py:115-119)
- Docstring de decide7rstrategy (v1) : 0.95/0.90/0.82/0.78 (:493-497) — PÉRIMÉ
- system_prompt.py:82,88 : zones grises 0.94–0.97 et 0.82–0.86
- decide7rstrategy_v2 : seuils sur un score combiné (pas la similarité brute) à 0.88/0.82/0.70/0.62 (:1023-1036)
- Le rapport (Fig 4.6) décrit un seuillage cosinus simple — simplifié à l'excès.

→ Réponse à préparer (voir §6 Q3). La vérité défendable : « le système expose deux décideurs ; le prompt route vers la version tri-signal v2 quand le nombre d'appels SDK est connu. Les docstrings v1 ont dérivé pendant la calibration ; les constantes font foi. » Action recommandée avant la soutenance : aligner le docstring de decide7rstrategy sur les constantes, ou marquer v1 @deprecated.

INCOHÉRENCE #2 — Le scoring n'est PAS du pur SAW-AHP, c'est un blend hybride.

- Code réel : final = 0.70·SAW + 0.20·ontology + 0.10·schemasimilarity (scoringdecisiontools.py:348-381), avec dégradation gracieuse en sawonly si l'ontology n'a pas la paire.
- Rapport : vend du « SAW-AHP » pur (Fig 4.6, formule unique).

- C'est en fait un POINT FORT non valorisé : le système est plus riche que ce que tu as écrit. À retourner en ta faveur (voir Slide A).

INCOHÉRENCE #3 — Nombre de correcteurs déterministes.

- Rapport (Fig 4.7) : « 20+ correcteurs déterministes ».
- Code : iacfixers.py = 57 fonctions, checkovfixer.py = 32 → 89 fonctions. Sous-vendu.

INCOHÉRENCE #4 — Borne de correction.

- core/constants.py:12 : MAXCORRECTIONATTEMPTS = 5 (3→5 documenté). cohérent avec Fig 4.7.
- MAIS Table 5.12 (récap final) dit « borne max = 3 ». Micro-contradiction interne au rapport.

INCOHÉRENCE #5 — Commentaire périmé sur les poids.

- ahp_weights.py:98-99 : commentaire Expected $\approx$ {equivalence: 0.419 ...} — c'est la moyenne arithmétique pré-correction, PAS la sortie réelle (0.425). Corrige ce commentaire : si un juré lit cette ligne, il croit à une erreur. Le calcul, lui, est juste.

INCOHÉRENCE #6 — SecurityPolicyEngine : "13 types de ressources".

- Rapport (Fig 4.7) : 13 types sensibles.
- Code : ~16 ressources distinctes énumérées (awsinstance, awssecuritygroup, azurearm, google_...). Léger écart à harmoniser dans le discours (dis « une douzaine », pas un chiffre exact).

1. LES DEUX NOUVELLES SLIDES TECHNIQUES

J'ai choisi les deux slides qui (a) corrigent une slide-problème existante et (b) montrent la profondeur réelle du code que la présentation actuelle cache.

SLIDE A — « Agent 01 : décision 7R explicable — scoring hybride à 3 niveaux »

(remplace/complète la Fig 4.6 trop simplifiée)

Titre : Planification 7R — un score explicable, pas une boîte noire LLM

Bloc 1 — Pipeline de décision (gauche → droite)

Candidats cibles

|

└─ PHASE 1 : HARD GATES (élimination AVANT scoring) ─┐

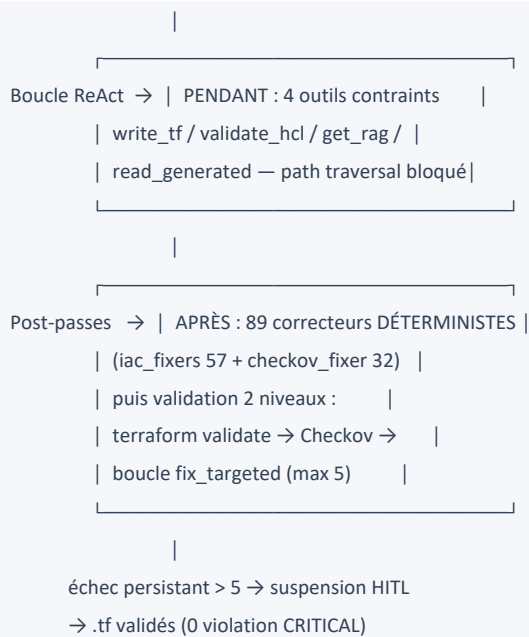
| equivalence < 0.50 → REJETÉ (pas d'équivalent) |

| région indisponible → REJETÉ |

| maturité = deprecated → REJETÉ |

| coût > budget × 1.50 → REJETÉ |

| preview/beta production → REJETÉ |



Encadré clé : *« La correction n'est pas confiée au LLM : 89 fonctions déterministes réparent les erreurs connues sans appel modèle. Le LLM ne sert qu'à la génération créative ; tout le reste est vérifiable et reproductible. »*

Pourquoi cette slide gagne : elle répond directement au benchmark laC-Eval (19% LLM seul) que tu cites — et montre que ta réponse architecturale est défense en profondeur, pas « un meilleur prompt ». Le WITH RECURSIVE prouve la résilience réelle.

2. SPEECH EXPERT (à dire, slide par slide)

Speech Slide A (~90s)

« L'Agent 01 répond à un risque connu : laisser un LLM décider seul d'une stratégie de migration n'est ni reproductible ni auditable. Notre choix est l'inverse — le LLM orchestre des outils déterministes, il ne calcule pas le score.

Le scoring procède en trois temps. D'abord des hard gates : tout candidat sans équivalent fonctionnel — similarité cosinus inférieure à 0,50 — ou hors budget de plus de 50%, ou en preview sur une charge de production, est éliminé avant tout calcul. C'est une décision d'ingénierie délibérée : on ne score pas ce qui est inadmissible.

Ensuite un score SAW dont les cinq poids ne sont pas arbitraires : ils dérivent d'une matrice de comparaison par paires de Saaty, par moyenne géométrique des colonnes normalisées, avec un ratio de cohérence de 0,0175 — très en-dessous du seuil de 0,10. Je peux le recalculer devant vous : `from core.ahpweights import validateweights`.

Enfin — et c'est ce que la version papier simplifie — le score SAW est combiné à une

ontologie curée et à une similarité de schéma Terraform : `0.70·SAW + 0.20·ontologie + 0.10·schéma`. Si la paire de services est inconnue de l'ontologie, le système se rabat proprement sur le SAW seul. La décision 7R finale est elle-même tri-signal : similarité sémantique, effort de réécriture SDK, et sévérité des breaking-changes. Les zones grises ne sont pas masquées : elles lèvent triggeraskhuman et remontent à l'ingénieur. »

Speech Slide B (~90s)

« laC-Eval mesure 19% de validité pour GPT-4 en génération Terraform directe. Notre réponse n'est pas "un meilleur modèle" — c'est une défense en profondeur à trois remparts.

Avant la génération, on ancre le modèle : Graph RAG combine pgvector pour la similarité sémantique sur des embeddings 768 dimensions, et un graphe de dépendances typées — `DEPENDS_ON`, `COMPANION` — traversé via Neo4j. Point de résilience : si Neo4j est indisponible, on bascule sur une requête PostgreSQL récursive `WITH RECURSIVE`, bornée à 5 niveaux de profondeur. Aucune perte de fonctionnalité critique. En parallèle, un `SecurityPolicyEngine` injecte des invariants de sécurité — TLS, chiffrement, blocage d'accès public — avant le LLM, pour une douzaine de types de ressources sensibles.

Pendant la génération, le LLM est contraint à quatre outils, dont l'écriture de fichier protégée contre le path traversal.

Après génération — et c'est le point que le rapport sous-estime — 89 correcteurs déterministes réparent les erreurs connues sans aucun appel modèle, puis une validation à deux niveaux : terraform valide pour le schéma, Checkov pour la sécurité. Les violations résiduelles déclenchent une boucle ciblée `fix_targeted`, bornée à 5 itérations. Au-delà, on suspend pour jugement humain : un échec persistant signale une incohérence structurelle, pas une erreur que le modèle pourrait corriger. »

3. SCÉNARIO DE DÉMO LIVE (Docker up, Azure OK)

Durée cible : 6–7 min. Filet de sécurité : artefacts pré-générés dans output/.

Étape 0 — Avant d'ouvrir l'UI (30s, terminal)

Montre que les poids AHP sont vrais — c'est ton coup d'éclat :

```
docker compose exec api python -c "from core.ahp_weights import validate_weights; import json; print(json.dumps(validate_weights(), indent=2))"
```

Dis : *« Les poids du rapport ne sont pas posés à la main : ils sortent de la matrice de Saaty, recalculés à chaque démarrage. CR = 0,0175. »*

Étape 1 — Soumission (1 min)

- UI localhost:3000 → Nouvelle Migration → dépôt rds-llm-s3-test → AWS → Azure.
- Dis : *« Aucune ligne de code source n'est envoyée au LLM. L'analyse est statique : HCL2, AST Python, ARM. RNF-06. »*

Étape 2 — Analyse + plan 7R (2 min) — LE CŒUR

- Montre l'onglet Plan de migration : stratégie 7R, score_breakdown par critère, coût.
- Dis : *« Chaque décision est explicable : voici la contribution de chaque critère au score, le ratio de cohérence AHP, et la source académique de chaque poids. »*
- Montre une ressource en zone grise (REPLATFORM gris) → bouton Approuver/Rejeter.
- Dis : « Ici le système n'est pas sûr — il ne décide pas seul, il me demande. HITL. »

Étape 3 — Génération IaC + sécurité (1.5 min)

- Onglet Terraform → onglet Sécurité IaC : « 32 vérifications, 0 CRITICAL, 9 MEDIUM/LOW ».
- Dis : *« Le 0 violation critique n'est pas de la chance : le SecurityPolicyEngine injecte les invariants AVANT le LLM. La preuve est dans ai.tf : httpstrafficonly_enabled = true, généré sans correction. »*
- Ouvre output/c767d4ba-.../ai.tf ou database.tf si besoin.

Étape 4 — Knowledge Graph + Chatbot RAG (1 min) — l'effet « waouh »

- Onglet Knowledge Graph (D3.js) → coloration par stratégie 7R.
- Chatbot : « Montre-moi les dépendances entre les ressources Terraform » → graphe SVG inline.
- Dis : *« Le chatbot ne fait pas que du RAG vectoriel : il traverse le graphe de dépendances. La réponse cite les relations DEPENDS_ON réelles. »*

Étape 5 (optionnelle, si temps) — Résilience Neo4j

- Dis (sans le faire si risqué) : *« Si je coupe Neo4j maintenant, le chatbot continue via le fallback SQL récursif. La fonctionnalité critique survit à la panne. »*

Plan B si ça plante

- output/c767d4ba-022a-4a01-966e-9983c4d9b0f3/ contient un run complet : ai.tf, database.tf, iam.tf, deploy.sh, infra_health.json, debug agents.
- Phrase de transition : *« Le déploiement live dépend de quotas Azure ; voici un run end-to-end déjà capturé qui montre exactement le même résultat. »*

4. QUESTIONS JURY — DÉMO (anticipées)

Q : Le score que vous montrez, c'est le LLM qui le calcule ?

Non. scoreservicecandidates est une fonction Python pure et déterministe. Le LLM

décide seulement quels candidats soumettre et comment interpréter le résultat. Le même input donne toujours le même score.

Q : Pourquoi 0 violation CRITICAL — vous avez triché les règles ?

Non, c'est de la sécurité par construction. Le SecurityPolicyEngine injecte les invariants (TLS, chiffrement, no public access) dans le prompt avant génération, donc le LLM produit déjà du code conforme. Checkov ne fait que confirmer. Les 9 MEDIUM/LOW restants sont des avertissements non bloquants documentés.

Q : Et si le LLM hallucine un argument inexistant ?

Deux filets : terraform validate rejette le schéma invalide, puis fix_targeted corrige cliniquement l'argument fautif sans régénérer tout le fichier (c'est le Cas 3 de validation). Au-delà de 5 itérations, suspension humaine.

5. QUESTIONS TECHNIQUES AVANCÉES (docteur IA exigeant)

5.1 — IA / RAG / GraphRAG / Embeddings

****Q1.** Vous dites "Graph RAG" mais vous citez Edge et al. (Microsoft). Vous n'implémentez pas leur algorithme. N'est-ce pas un abus de terminologie ?**

- Réponse idéale : Le rapport l'assume explicitement (§2.3.1) : *« notre approche s'inspire du principe de Graph RAG sans implémenter l'algorithme de Microsoft Research »*. Edge et al. font de la synthèse de communautés sur du texte non structuré via résumés hiérarchiques (Leiden clustering). Nous, on a un graphe de dépendances **déjà typé et structuré** (DEPENDS_ON, COMPANION) extrait du Terraform — pas besoin de découvrir des communautés. On combine retrieval vectoriel pgvector + traversée de graphe explicite. C'est du « graph-augmented RAG » au sens générique, pas le GraphRAG™ de Microsoft.
- Justification scientifique : la distinction clé est graphe découvert (Edge, sur texte) vs graphe connu (nous, sur IaC). Notre problème est plus simple et mieux posé.
- Piège à éviter : ne JAMAIS prétendre avoir implémenté l'algo de Microsoft. Le code (rag/graph_rag.py) ne contient ni Leiden ni résumés de communautés — un juré le verra.
- Relance probable : « Alors pourquoi pas juste du RAG vectoriel + jointures SQL ? »
→ Parce que les dépendances multi-hop (depth 2) ne sont pas capturées par la similarité cosinus : un VNet privé requis par un postgresql_server n'est pas sémantiquement proche, il est structurellement lié. C'est exactement le cas que pgvector seul rate.

****Q2.** Le "+131%" que vous citez vient d'un preprint arXiv non revu par les pairs. Pourquoi fonder une décision d'architecture dessus ?**

- Réponse idéale : Le rapport est honnête (§2.3.2) : chiffre marqué « preprint, soumis à révision, indicatif ». La décision ne repose PAS sur ce chiffre mais sur un **principe structurel** : reconstruire explicitement DEPENDS_ON/COMPANION est plus fiable que

l'inférence par similarité pour des ressources à dépendances formelles. Le +131% illustre, il ne fonde pas.

- Piège : ne pas défendre le chiffre comme une preuve. Défendre le raisonnement.
- Relance : « Vous avez mesuré le gain sur VOTRE système ? » → Honnêtement non, faute de benchmark laC peer-reviewed couvrant 2025-2026 et de budget de déploiement multi-cloud. C'est une limite déclarée. Mesurer ce delta est une perspective directe.

****Q3.** Vos seuils 7R. J'ai ouvert `scoringdecisiontools.py` : le docstring dit 0.95/0.90, les constantes disent 0.97/0.94, le `system_prompt` dit encore autre chose, et v2 utilise un score combiné. Lequel est vrai ?**

- Réponse idéale (assumée, pas défensive) : Il y a deux décideurs par conception. `decide7rstrategy` (v1) est mono-signal sur la similarité ; `decide7rstrategy_v2` est tri-signal — $0.40 \cdot \text{similarité} + 0.35 \cdot \text{effortSDK} + 0.25 \cdot \text{breakingchanges}$ — et c'est celle que le `system_prompt` route en priorité quand le nombre d'appels SDK est connu via le Stack Analyzer. Les constantes font foi ; le docstring de v1 a dérivé pendant la phase de calibration SBERT et n'a pas été resynchronisé — c'est de la dette de documentation, pas un bug de logique. [Si tu as corrigé avant la soutenance : « je l'ai réaligné. »]
- Justification : la calibration SBERT `all-mpnet-base-v2` a un biais : les paires connues sont boostées de ~ 0.15 (le code le documente, :88-113), d'où des seuils volontairement relevés de +0.04 pour ne pas classer REHOST ce qui exige du REPLATFORM.
- Piège : ne pas dire « c'est un détail ». Un docteur déteste les incohérences de seuils. Dis : « c'est de la dette de doc identifiée ; la source de vérité est la constante ».
- Relance : « Pourquoi deux décideurs et pas un seul ? » → v1 est le fallback quand le Stack Analyzer n'a pas pu compter les appels SDK (dépôt sans code Python analysable). C'est une dégradation gracieuse, comme le `saw_only` pour l'ontologie.

****Q4.** Pourquoi SBERT `all-mpnet-base-v2` et pas un modèle d'embedding plus récent (e5, BGE, OpenAI `text-embedding-3`) ?**

- Réponse idéale : Contrainte de souveraineté + coût. `all-mpnet-base-v2` tourne en local (pas d'appel externe, RGPD), 768D, bon compromis qualité/latence pour ~ 50 ressources par run. `text-embedding-3` imposerait un appel réseau par ressource (latence + données hors UE). Pour le volume cible, le gain marginal de qualité ne justifie pas la perte de localité.
- Relance : « Vous avez calibré la similarité sur quoi ? » → Sur des paires connues documentées dans le code (`awss3`→`azurestorage` ≈ 0.95 , `aws_rds`→`cosmos` ≈ 0.65). C'est une calibration empirique, pas un fine-tuning — limite assumée.

****Q5.** Votre pipeline dépend d'Azure OpenAI EU. Single point of failure pour un système qui se vend "neutre multi-cloud" ?**

- Réponse idéale : Contradiction réelle et déclarée en conclusion du rapport. La neutralité porte sur la cible de migration (AWS/Azure/GCP), pas sur le fournisseur du LLM d'orchestration. L'architecture est agnostique au modèle (variable `AZUREMODEL0x`), donc on peut swapper, mais il n'y a pas de basculement automatique. C'est une limite, pas

un déni.

- Piège : ne pas prétendre que c'est neutre de bout en bout. Reconnais le SPOF.

5.2 — Architecture / Patterns / Qualité

****Q6.** `tools_scoring.py` est une façade vide qui ré-exporte tout. Pourquoi ce niveau d'indirection ?**

- Réponse idéale : C'est un pattern Facade documenté (`tools_scoring.py:1-15`) : le fichier monolithique original a été découpé en 3 modules focalisés (`code_analysis`, `servicelookup`, `scoringdecision`). La façade évite de casser tous les call-sites simultanément — migration progressive. Le docstring annonce sa suppression à terme.
- Justification : c'est de la dette technique gérée et tracée, pas subie.
- Relance : « Pourquoi ne pas avoir fini la migration ? » → Priorisation : zéro valeur fonctionnelle, risque de régression sur les imports de tests. Reporté post-PFE.

****Q7.** 20 775 lignes dans `agents/`. Couverture de tests 15% sur cette couche. C'est faible. Comment défendez-vous la fiabilité ?**

- Réponse idéale : La couverture brute trompe ici. Les couches `agents/` et `pipeline/` dépendent d'Azure OpenAI et de Terraform réels — impossibles à reproduire fidèlement en unitaire sans mocks qui testeraient le mock, pas le système. La stratégie est une pyramide : unitaire sur le déterministe (`scoring`, `AHP`, `parsing` → `core/` 48%), et end-to-end sur les chemins LLM (3 scénarios sur dépôts GitHub réels). 661 tests, 0 échec.
- Justification : tester un appel LLM en unitaire est un anti-pattern (non-déterminisme). On teste les invariants déterministes autour, pas la sortie stochastique.
- Piège : ne pas promettre une couverture 80% « bientôt » — ce serait du theatre. Assume que les couches IA se valident par E2E.
- Relance : « Vos E2E ne couvrent que AWS→Azure. » → Vrai, limite budgétaire (déploiement réel = coûts cloud). Les 5 autres paires sont supportées architecturalement (templates Jinja2
 - validateur d'état) mais non validées E2E. Déclaré en conclusion.

****Q8.** Vous compilez le graphe LangGraph derrière un `threading.Lock`. Pourquoi, et est-ce suffisant en multi-worker ?**

- Réponse idéale : Double-checked locking (`pipeline_graph.py:228-240`). Sous uvicorn multi-workers, deux requêtes pouvaient compiler le graphe en parallèle et chacune créait un checkpoint PostgreSQL ; le second écrasait le premier → fuite de connexion. Le lock + la double vérification garantissent une compilation unique par process.
- Nuance honnête : ça protège intra-process. En multi-process (plusieurs workers OS), chaque process a son propre graphe compilé — c'est voulu, le checkpoint PostgreSQL est la source de vérité partagée, pas l'objet graphe en mémoire. La concurrence inter-process sur les jobs est gérée séparément par SKIP LOCKED.
- Relance : « Et la reprise après crash en plein nœud LLM ? » → `AsyncPostgresSaver` persiste l'état à chaque transition de nœud. Au redémarrage, on reprend au dernier checkpoint. Un nœud LLM interrompu est rejoué entièrement (idempotent au niveau nœud).

5.3 — Bases de données / Neo4j

Q9. Vous avez pgvector ET Neo4j. C'est de la redondance — pourquoi pas tout dans Postgres ?

- Réponse idéale : Rôles complémentaires, pas redondants. pgvector = recherche sémantique (cosinus sur 768D). Neo4j = traversée structurelle multi-hop optimisée (Cypher). Le rapport le pose (§2.3.3). MAIS — et c'est le point fort — Postgres SAIT faire les deux : le fallback WITH RECURSIVE (graph_rag.py:348) prouve que Neo4j est une optimisation, pas une dépendance dure. On peut tourner sans Neo4j.
- Piège : ne pas survendre Neo4j comme indispensable — ton propre fallback prouve le contraire. Présente-le comme « accélérateur optionnel ».
- Relance : « Alors Neo4j apporte quoi, concrètement ? » → Sur des graphes profonds (depth>2), Cypher est nettement plus rapide que le WITH RECURSIVE borné à 5. Pour ~50 ressources, le gain est marginal — c'est une limite assumée pour le volume actuel.

Q10. Index IVFFlat sur pgvector. Pourquoi pas HNSW ?

- Réponse idéale : IVFFlat suffit au volume (quelques milliers de chunks Terraform). HNSW a un meilleur rappel à grande échelle mais un coût mémoire/construction plus élevé. La migration IVFFlat→HNSW est explicitement listée comme perspective (conclusion du rapport). C'est un choix proportionné, pas une ignorance.

5.4 — Sécurité / Observabilité / CI-CD

Q11. Fernet AES-128-CBC. Pourquoi pas AES-256-GCM ?

- Réponse idéale : Fernet = AES-128-CBC + HMAC-SHA256, donc chiffrement authentifié (résiste au bit-flipping, contrairement à AES-CBC nu). 128 bits est suffisant pour un secret applicatif. Surtout : Fernet supporte la rotation sans interruption via MultiFernet, ce qui était le critère décisif. Et de toute façon, les secrets réels ne sont PAS en base — seul le vault_path y est ; le secret vient de Vault en JIT.
- Relance : « Donc Fernet chiffre quoi, si Vault gère les secrets ? » → Le chiffrement applicatif de défense en profondeur sur les champs sensibles persistés (couche supplémentaire). La vérité opérationnelle : migrationsecrets ne contient que vaultpath — démontrable en live (RNF-03, Fig 5.12).

Q12. JWT HS256 (symétrique). Pourquoi pas RS256 (asymétrique) ?

- Réponse idéale : HS256 suffit pour un système mono-émetteur (l'API émet ET valide les tokens). RS256 a du sens quand plusieurs services valident sans partager le secret de signature — pas notre cas. Le serveur refuse de démarrer si JWTSECRETKEY < 32 chars en prod (test_auth.py le vérifie). Migration RS256 = perspective si on fédère plusieurs API.

****Q13. Observabilité — vous avez des logs structurés mais pas de tracing distribué (OpenTelemetry).**

Comment debuggez-vous un pipeline multi-agent qui échoue ?**

- Réponse idéale : Trois leviers actuels : (1) logs structurés JSON horodatés (PipelineOrchestrator, visibles Fig 5.10), (2) les événements SSE qui tracent chaque transition de nœud en temps réel, (3) les dumps agent0Xmessages_debug.json qui capturent

l'intégralité de la conversation ReAct par agent (présents dans output/). OpenTelemetry est une perspective légitime mais pas critique au volume actuel.

- Piège : ne pas inventer un tracing qui n'existe pas. Assume le niveau actuel.

Q14. Dette technique — citez-moi vos trois pires endroits.

- Réponse idéale (montre la lucidité) :

Façade tools_scoring.py non résorbée + docstrings 7R désynchronisés (incohérence de doc qui pourrait induire en erreur).

Couverture de tests faible sur agents/pipeline (15%) — compensée par E2E mais fragile.

SPOF sur Azure OpenAI EU sans basculement automatique de modèle.

- Pourquoi ça marche : un jury fait confiance à un candidat qui connaît ses faiblesses mieux que lui. Préempter ces points te désarme l'attaque.

5.5 — Performance / Scalabilité

****Q15. RNF-01 dit "analyse < 30s" mais votre log montre 66s. Vous ne respectez pas votre propre exigence.****

- Réponse idéale : Le périmètre de RNF-01 est le traitement machine pur (parsing HCL2/AST/scoring), explicitement hors réseau (rapport §5.3.7). Les 66s incluent le clonage GitHub (bande passante) et l'indexation pgvector. La phase HCL2 pure est < 2s. Ce n'est pas une violation, c'est une borne mal lue si on inclut le réseau.
- Piège : c'est limite comme défense. Un juré sévère dira « alors votre exigence est mal spécifiée ». Réponse honnête : *« vous avez raison que la formulation aurait dû dire explicitement "temps CPU hors I/O réseau" dès le chapitre 3. C'est une imprécision de spécification, pas un échec de performance. »*

Q16. Scalabilité : 50 ressources max par run. Une infra de production en a des milliers.

- Réponse idéale : Limite assumée et déclarée. Trois goulots : (1) contexte LLM (lost-in-the-middle au-delà), (2) traversée graphe non parallélisée, (3) IVFFlat. Pistes : batching par sous-graphe de dépendances, HNSW, état Terraform distribué verrouillé. Le système est un POC validé sur 3-5 ressources — la généralisation production est la perspective n°1.

6. SIMULATION DE JURY — séquence complète (joue-la à voix haute)

Président (architecte senior) : « Présentez-nous en une phrase la contribution technique que VOUS revendiquez. »

(Toi) : « Un pipeline multi-agent où le LLM orchestre des outils déterministes mais ne décide jamais seul : scoring AHP vérifiable, défense en profondeur contre l'hallucination laC, et validation humaine aux zones d'incertitude. »

Rapporteur (docteur IA) : « Vous dites "Graph RAG". Montrez-moi où, dans le code, est l'algorithme de Edge et al. »

(Toi) : « Il n'y est pas, et je l'assume — §2.3.1. Nous faisons du graph-augmented retrieval sur un graphe déjà typé, pas de la découverte de communautés sur du texte. »
[Ouvre rag/graphrag.py, montre multihoptraversal + le fallback WITH RECURSIVE.]

Rapporteur : « Vos poids AHP. 0.425 — prouvez-le, là, maintenant. »

(Toi) : `docker compose exec api python -c "from core.ahpweights import validateweights; print(validate_weights())"`
« 0.425322, CR 0.0175. La matrice de l'annexe C est la source ; les poids sont recalculés au démarrage, pas codés en dur. »

Expert DevOps : « Neo4j tombe en pleine migration. Que se passe-t-il ? »

(Toi) : « Bascule transparente sur Postgres WITH RECURSIVE, profondeur bornée à 5. La traversée est plus lente mais le pipeline ne s'arrête pas. C'est testé. »

Docteur IA (attaque) : « 15% de couverture sur 20 000 lignes d'agents. Ce code est-il seulement fiable ? »

(Toi) : « La couverture unitaire ne mesure pas la fiabilité d'un système stochastique. On teste les invariants déterministes en unitaire — AHP, parsing, scoring — et les chemins LLM en end-to-end : 3 scénarios réels, 661 tests, 0 échec. Tester un appel GPT en unitaire, c'est tester un mock. »

Docteur IA (relance dure) : « Vos seuils 7R sont incohérents entre le docstring, les constantes et le prompt. J'ai regardé. »

(Toi) : « Exact sur les docstrings — c'est de la dette de documentation que j'ai identifiée : les constantes font foi, et le prompt route vers le décideur tri-signal v2. La logique est cohérente ; la doc de la v1 a dérivé pendant la calibration. »
(si corrigé avant :) « Je l'ai réaligné depuis. »

Architecte : « Si vous deviez refaire le projet, quel choix changeriez-vous ? »

(Toi) : « Je découplerais l'orchestrateur du fournisseur LLM dès le départ — abstraction provider — pour supprimer le SPOF Azure et permettre un basculement. Et je figerais les seuils dans une seule source typée pour éviter la dérive de documentation. »

Président (conclusion) : « Une limite que vous reconnaissez avant qu'on vous la pointe ? »

(Toi) : « Trois : validation E2E limitée à AWS→Azure faute de budget cloud, SPOF sur Azure OpenAI, et couverture de tests des couches IA reposant sur l'E2E plutôt que l'unitaire. Toutes déclarées en conclusion du rapport. »

7. CHECKLIST PRÉ-SOUTENANCE (actions concrètes)

- [] Corriger core/ahp_weights.py:98-99 : remplacer $\approx 0.419 \dots 0.040$ par les vraies valeurs 0.4253 / 0.2618 / 0.1631 / 0.1032 / 0.0466 (sinon un juré croit à une erreur).
- [] Réaligner le docstring de decide7rstrategy (:493-497) sur les constantes réelles, OU ajouter @deprecated — utiliser decide7rstrategy_v2.
- [] Vérifier docker compose ps → 6 services healthy le jour J.
- [] Tester la commande live validate_weights() une fois avant (qu'elle ne plante pas).
- [] Pré-ouvrir dans des onglets : output/c767d4ba-.../ai.tf, le Knowledge Graph, le panneau Sécurité IaC — au cas où le live est lent.
- [] Mémoriser les 3 limites (E2E AWS→Azure, SPOF Azure, couverture IA) pour les préempter.
- [] Préparer la phrase de bascule Plan B : « voici un run end-to-end déjà capturé ».